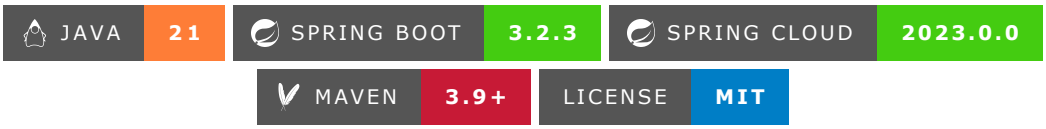
README

🎓 Sistema de Gestión Académica - Microservicios



Sistema académico distribuido que demuestra dominio de POO, principios SOLID, patrones de diseño y arquitectura de microservicios.

📋 Tabla de Contenidos

- Descripción del Proyecto
- Arquitectura
- Estructura del Proyecto
- Tecnologías
- Patrones de Diseño
- Principios SOLID
- Requisitos Previos
- Instalación
- Ejecución
- API Endpoints
- Testing
- Frontend
- Docker
- Documentación
- Autores

📖 Descripción del Proyecto

Este proyecto implementa un **Sistema de Gestión Académica** utilizando una arquitectura de **microservicios**. El sistema permite gestionar:

- **Estudiantes**: Registro, consulta y gestión de estados (ACTIVE, INACTIVE, GRADUATED, SUSPENDED)

- **Cursos**: Creación y administración de cursos académicos con capacidad limitada
- **Matrículas**: Inscripción de estudiantes en cursos con validaciones de negocio

Características Principales

- ✅ Arquitectura de microservicios con **Spring Cloud Gateway**
- ✅ Comunicación síncrona entre servicios vía **REST**
- ✅ Patrones de diseño: **Strategy** y **Template Method**
- ✅ Aplicación de principios **SOLID**
- ✅ Base de datos **H2** en memoria por servicio
- ✅ **36 tests** (19 unitarios + 17 de integración)
- ✅ Colección **Postman** con 12 requests
- ✅ Documentación con **diagramas UML** (PlantUML)
- ✅ Frontend de ejemplo con **React + Vite + Tailwind CSS**
- ✅ Configuración **Docker** lista para despliegue
- ✅ Scripts de gestión para iniciar/detener servicios

🏗️ Arquitectura

Diagrama de Arquitectura

```
┌──────────────────────────────────────────────────────────────────────────────
│                                CLIENTES
│
│                       (Navegador / Postman / Frontend React)
│
└──────────────────────────────────────────────────────────────────────────────
                                      │
                                      │ HTTP
                                      ▼
┌──────────────────────────────────────────────────────────────────────────────
│                            API GATEWAY (:8080)
│
│
┌──────────────────────────────────────────────────────────────────────────────
│
│   │   • Punto de entrada único
│     │
│   │   • Enrutamiento inteligente
│     │
│   │   • Correlation-ID para trazabilidad
│     │
│   │   • Logging centralizado
│     │
│
```

```
└──────────────
```

└──

Tabla de Microservicios

Servicio	Puerto	Base de Datos	Descripción
api-gateway	8080	-	Punto de entrada único, enrutamiento y correlation-ID
ms-estudiantes	8081	`jdbc:h2:mem:estudiantesdb`	CRUD de estudiantes y gestión de estados
ms-cursos	8082	`jdbc:h2:mem:cursosdb`	CRUD de cursos académicos
ms-matriculas	8083	`jdbc:h2:mem:matriculasdb`	Orquestador de matrículas con validaciones

Comunicación entre Servicios

```
┌──────────────────┐                          ┌──────────────────────┐
│  MS-MATRICULAS   │ ──────── GET ──────────► │  MS-ESTUDIANTES      │
│                  │      /students/{id}      │   Valida estado      │
│   (Orquestador)  │                          └──────────────────────┘
│                  │
│                  │ ──────── GET ──────────► ┌──────────────────────┐
│                  │      /courses/{id}       │    MS-CURSOS         │
└──────────────────┘                          │ Valida capacidad     │
                                              └──────────────────────┘
```

📁 Estructura del Proyecto

```
poo-project/
│
├── 📄 pom.xml                          # POM padre (Maven multi-
módulo)
├── 📄 README.md                        # Este archivo
├── 📄 PLANNING.md                      # Documento de planificación
├── 📄 docker-compose.yml               # Orquestación Docker
├── 📄 .dockerignore                    # Archivos ignorados por Docker
│
├── 📂 common/                          # Módulo común compartido
│   ├── pom.xml
│   └── src/main/java/com/academic/common/
│       ├── dto/                        # DTOs compartidos
│       ├── exception/                  # Excepciones base
│       └── util/                       # Utilidades comunes
```

```
│
├── 📂 api-gateway/                    # API Gateway (Spring Cloud
Gateway)
│   ├── pom.xml
│   ├── Dockerfile
│   └── src/main/
│       ├── java/com/academic/gateway/
│       │   ├── GatewayApplication.java
│       │   ├── config/                # Configuración del Gateway
│       │   └── filter/                # Filtros personalizados
│       └── resources/
│           └── application.yml        # Rutas y configuración
│
├── 📂 ms-estudiantes/                 # Microservicio de Estudiantes
│   ├── pom.xml
│   ├── Dockerfile
│   └── src/
│       ├── main/java/com/academic/estudiantes/
│       │   ├── EstudiantesApplication.java
│       │   ├── controller/            # REST Controllers
│       │   ├── service/               # Lógica de negocio
│       │   ├── repository/            # Spring Data JPA
│       │   ├── domain/
│       │   │   ├── entity/            # Entidades JPA
│       │   │   ├── dto/               # Request/Response DTOs
│       │   │   └── enums/             # Enumeraciones
│       │   ├── mapper/                # MapStruct mappers
│       │   └── config/                # Configuraciones
│       └── test/                      # Tests unitarios e integración
│
├── 📂 ms-cursos/                      # Microservicio de Cursos
│   ├── pom.xml
│   ├── Dockerfile
│   └── src/                           # (Estructura similar a ms-
estudiantes)
│
├── 📂 ms-matriculas/                  # Microservicio de Matrículas
│   ├── pom.xml
│   ├── Dockerfile
│   └── src/main/java/com/academic/matriculas/
│       ├── MatriculasApplication.java
│       ├── controller/
│       ├── service/
│       │   └── impl/
│       │       └── EnrollmentServiceImpl.java  # Template Method
│       ├── repository/
│       ├── domain/
│       ├── mapper/
│       ├── client/                    # REST Clients (Feign-like)
│       │   ├── StudentServiceClient.java
│       │   └── CourseServiceClient.java
│
```

```
│       ├── validation/                # Patrón Strategy
│       │   ├── EnrollmentValidationStrategy.java
│       │   └── impl/
│       │       ├── StudentActiveValidation.java
│       │       ├── CourseActiveValidation.java
│       │       ├── CourseCapacityValidation.java
│       │       └── DuplicateEnrollmentValidation.java
│       └── config/
│
├── 📂 frontend-example/              # Frontend React (opcional)
│   ├── package.json
│   ├── vite.config.js
│   └── src/
│       ├── App.jsx
│       ├── api/index.js              # Cliente API
│       └── components/
│           ├── Students.jsx
│           ├── Courses.jsx
│           └── Enrollments.jsx
│
├── 📂 docs/                          # Documentación
│   ├── DESIGN.md                     # Documento de diseño técnico
│   ├── SUSTENTACION.md               # Guía de sustentación
│   ├── diagrams/
│   │   ├── architecture.puml         # Diagrama de arquitectura
│   │   ├── class-diagram.puml        # Diagrama de clases
│   │   ├── sequence-diagram.puml     # Diagrama de secuencia
│   │   └── output/                   # PNGs generados
│   ├── postman/
│   │   └── academic-system.postman_collection.json
│   └── pdf/                          # Documentación HTML exportada
│
├── 📂 logs/                          # Logs de los microservicios
│
└── 📜 Scripts de gestión
    ├── start-all.sh                  # Iniciar todos los servicios
    ├── stop-all.sh                   # Detener todos los servicios
    ├── restart-all.sh                # Reiniciar todos los servicios
    └── status.sh                     # Ver estado de los servicios
```

🛠️ Tecnologías

Backend

Tecnología	Versión	Propósito
Java	21 LTS	Lenguaje principal
Spring Boot	3.2.3	Framework base

Tecnología	Versión	Propósito
Spring Cloud Gateway	2023.0.0	API Gateway reactivo
Spring Data JPA	3.2.x	Persistencia ORM
Spring Validation	3.2.x	Validación de DTOs
Spring Actuator	3.2.x	Health checks y métricas
H2 Database	2.2.x	Base de datos en memoria
MapStruct	1.5.5	Mapeo Entity ↔ DTO
Lombok	1.18.x	Reducción de boilerplate
Maven	3.9+	Gestión de dependencias

Testing

Tecnología	Propósito
JUnit 5	Framework de testing
Mockito	Mocking de dependencias
Spring Boot Test	Testing de integración
JaCoCo	Cobertura de código

Frontend (Opcional)

Tecnología	Versión	Propósito
React	18.x	Librería UI
Vite	7.3.1	Build tool
Tailwind CSS	3.x	Estilos utilitarios

DevOps

Tecnología	Propósito
Docker	Contenedorización
Docker Compose	Orquestación local

🎨 Patrones de Diseño

1. Strategy Pattern (Patrón Estrategia)

Ubicación: `ms-matriculas/src/main/java/com/academic/matriculas/validation/`

El patrón Strategy se utiliza para las **validaciones de matrícula**. Cada regla de negocio está encapsulada en su propia clase, permitiendo agregar o modificar validaciones sin alterar el código existente.

```
// Interfaz Strategy
public interface EnrollmentValidationStrategy {
    void validate(EnrollmentCreateRequest request, StudentDTO student,
        CourseDTO course);
}

// Implementaciones concretas
├── StudentActiveValidation.java      // Valida que el estudiante esté
        activo
├── CourseActiveValidation.java       // Valida que el curso esté
        activo
├── CourseCapacityValidation.java     // Valida que haya cupos
        disponibles
└── DuplicateEnrollmentValidation.java // Valida que no exista
        matrícula duplicada
```

Diagrama:

```
            ┌──────────────────────────────┐
            │ EnrollmentValidationStrategy │ ◄── Interface
            │         <<interface>>        │
            └──────────────────────────────┘
                          ▲
                          │ implements
                          │
        ┌─────────────────┼─────────────────┬─────────────────┐
        │                 │                 │                 │
┌───────┴─────────┐ ┌─────┴──────┐ ┌────────┴────────┐ ┌──────┴──────┐
│StudentActive    │ │CourseActive│ │CourseCapacity   │ │Duplicate    │
│  Validation     │ │ Validation │ │  Validation     │ │ Enrollment  │
└─────────────────┘ └────────────┘ └─────────────────┘ └─────────────┘
```

Uso en el servicio:

```
@Service
public class EnrollmentServiceImpl {
    private final List<EnrollmentValidationStrategy>
        validationStrategies;

    private void executeValidations(EnrollmentCreateRequest request,
                                    StudentDTO student, CourseDTO
        course) {
        for (EnrollmentValidationStrategy strategy :
        validationStrategies) {
            strategy.validate(request, student, course);
        }
    }
}
```

2. Template Method Pattern (Patrón Método Plantilla)

Ubicación: `ms-matriculas/src/main/java/com/academic/matriculas/service/impl/EnrollmentServiceImpl.java`

El patrón Template Method define el **esqueleto del algoritmo** para crear una matrícula. Los pasos están definidos en orden y algunos pueden ser sobrescritos por subclases.

```java
/**
 * Template Method para crear matrícula:
 * 1. Obtener datos de servicios externos
 * 2. Ejecutar validaciones (Strategy Pattern)
 * 3. Crear la matrícula
 * 4. Post-procesamiento
 */
@Override
@Transactional
public EnrollmentResponse createEnrollment(EnrollmentCreateRequest
        request) {
    // Step 1: Obtener datos de servicios externos
    StudentDTO student = fetchStudentData(request.getStudentId());
    CourseDTO course = fetchCourseData(request.getCourseId());

    // Step 2: Ejecutar validaciones ordenadas (Strategy Pattern)
    executeValidations(request, student, course);

    // Step 3: Crear y persistir la matrícula
    Enrollment enrollment = createAndSaveEnrollment(request);

    // Step 4: Post-procesamiento (logging, eventos, etc.)
    postProcessEnrollment(enrollment, student, course);

    return enrollmentMapper.toResponse(enrollment);
}
```

Diagrama de Secuencia:

```
Client          Controller        Service           Clients
Repository
  |                 |                |                  |
|
  | POST /enrollments              |                  |
|
  |─────────────────►|                |                  |
|
  |                 | createEnrollment                 |
|
  |                 |─────────────────►|                  |
|
  |                 |                |                  |
|
  |                 |                | Step 1: fetchStudentData
|
  |                 |                |──────────────────►|
|
  |                 |                |◄──────────────────|
|
```

```
  |                  |                  |                   |
|
  |                  |                  | Step 1: fetchCourseData
|
  |                  |                  |──────────────────►|
|
  |                  |                  |◄──────────────────|
|
  |                  |                  |                   |
|
  |                  |                  | Step 2: executeValidations()
|
  |                  |                  | (Strategy Pattern - 4
validaciones)
  |                  |                  |                   |
|
  |                  |                  | Step 3: createAndSaveEnrollment
|
  |                  |
|──────────────────────────────────────►
  |                  |
|◄──────────────────────────────────────
  |                  |                  |                   |
|
  |                  |                  | Step 4: postProcessEnrollment
|
  |                  |                  |                   |
|
  |                  |◄─────────────────|                   |
|
  |◄─────────────────|                  |                   |
|
  |  201 Created     |                  |                   |
|
```

🔷 Principios SOLID

S - Single Responsibility Principle (SRP)

> "Una clase debe tener una única razón para cambiar"

Capa	Responsabilidad Única
`Controller`	Manejar requests/responses HTTP
`Service`	Lógica de negocio
`Repository`	Acceso a datos
`Mapper`	Transformación Entity ↔ DTO
`ValidationStrategy`	Una única regla de validación

Ejemplo:

```
// Cada estrategia tiene UNA sola responsabilidad
public class StudentActiveValidation implements
        EnrollmentValidationStrategy {
    @Override
    public void validate(...) {
        // SOLO valida que el estudiante esté activo
        if (student.getStatus() != StudentStatus.ACTIVE) {
            throw new BusinessException("Student is not active");
        }
    }
}
```

O - Open/Closed Principle (OCP)

> "Abierto para extensión, cerrado para modificación"

Para agregar una nueva validación de matrícula:

1. Crear nueva clase que implemente `EnrollmentValidationStrategy`
2. Anotarla con `@Component` y `@Order(n)`
3. **NO se modifica** código existente

```
// Nueva validación sin modificar código existente
@Component
@Order(5)
public class PrerequisiteValidation implements
        EnrollmentValidationStrategy {
    @Override
    public void validate(...) {
        // Nueva lógica de prerrequisitos
    }
}
```

L - Liskov Substitution Principle (LSP)

> "Los objetos de una superclase deben poder ser reemplazados por objetos de sus subclases"

Todas las implementaciones de `EnrollmentValidationStrategy` son **intercambiables**:

```
// El servicio no sabe qué implementación específica está usando
private final List<EnrollmentValidationStrategy> validationStrategies;

for (EnrollmentValidationStrategy strategy : validationStrategies) {
    strategy.validate(request, student, course);  // Polimorfismo
}
```

I - Interface Segregation Principle (ISP)

> "Los clientes no deben depender de interfaces que no usan"

Las interfaces están segregadas por dominio:

```
// Interfaces específicas por dominio
public interface StudentService { ... }
public interface CourseService { ... }
public interface EnrollmentService { ... }
public interface EnrollmentValidationStrategy { ... }  // Solo un
        método
```

D - Dependency Inversion Principle (DIP)

> "Depende de abstracciones, no de implementaciones"

```
@Service
public class EnrollmentServiceImpl implements EnrollmentService {
    // Depende de la INTERFAZ, no de implementaciones concretas
    private final List<EnrollmentValidationStrategy>
        validationStrategies;
    private final StudentServiceClient studentServiceClient;  //
        Interface
    private final CourseServiceClient courseServiceClient;    //
        Interface

    // Spring inyecta las implementaciones automáticamente
}
```

📋 Requisitos Previos

Software Requerido

Software	Versión Mínima	Verificar
JDK	21	`java --version`
Maven	3.9.x	`mvn --version`
Git	2.x	`git --version`
Node.js (opcional)	18.x	`node --version`
Docker (opcional)	20.x	`docker --version`

Verificar Instalación

```
# Java
java --version
# openjdk 21.0.x 2024-xx-xx LTS
```

```
# Maven
mvn --version
# Apache Maven 3.9.x

# Git
git --version
# git version 2.x.x
```

📥 Instalación

1. Clonar el Repositorio

```
git clone <url-repositorio>
cd poo-project
```

2. Compilar el Proyecto

```
# Compilar todos los módulos (incluye tests)
mvn clean install

# O sin tests (más rápido)
mvn clean install -DskipTests
```

3. Verificar Compilación

```
# Debe mostrar BUILD SUCCESS para los 5 módulos
[INFO] Reactor Summary:
[INFO] Academic System - Parent ........................... SUCCESS
[INFO] common ............................................ SUCCESS
[INFO] ms-estudiantes .................................... SUCCESS
[INFO] ms-cursos ......................................... SUCCESS
[INFO] ms-matriculas ..................................... SUCCESS
[INFO] api-gateway ....................................... SUCCESS
```

🚀 Ejecución

Opción 1: Scripts de Gestión (Recomendado)

```
# Dar permisos de ejecución (solo primera vez)
chmod +x *.sh

# Iniciar todos los servicios
./start-all.sh
```

```
# Ver estado de los servicios
./status.sh

# Detener todos los servicios
./stop-all.sh

# Reiniciar todos los servicios
./restart-all.sh
```

Opción 2: Ejecución Manual

Abrir 4 terminales y ejecutar en cada una:

```
# Terminal 1 - MS Estudiantes
cd ms-estudiantes && mvn spring-boot:run

# Terminal 2 - MS Cursos
cd ms-cursos && mvn spring-boot:run

# Terminal 3 - MS Matrículas
cd ms-matriculas && mvn spring-boot:run

# Terminal 4 - API Gateway
cd api-gateway && mvn spring-boot:run
```

Opción 3: Docker Compose

```
# Construir imágenes y levantar contenedores
docker-compose up --build

# En segundo plano
docker-compose up -d --build

# Detener
docker-compose down
```

Verificar que los Servicios están Corriendo

```
# Health checks individuales
curl http://localhost:8081/actuator/health  # MS-Estudiantes
curl http://localhost:8082/actuator/health  # MS-Cursos
curl http://localhost:8083/actuator/health  # MS-Matrículas
curl http://localhost:8080/actuator/health  # API Gateway

# O usar el script de estado
./status.sh
```

URLs de los Servicios

Servicio	URL	Descripción
API Gateway	http://localhost:8080	Punto de entrada principal
MS-Estudiantes	http://localhost:8081	Acceso directo
MS-Cursos	http://localhost:8082	Acceso directo
MS-Matrículas	http://localhost:8083	Acceso directo
Frontend React	http://localhost:5173	UI de prueba

Consolas H2 (Base de Datos)

Servicio	URL	JDBC URL
Estudiantes	http://localhost:8081/h2-console	`jdbc:h2:mem:estudiantesdb`
Cursos	http://localhost:8082/h2-console	`jdbc:h2:mem:cursosdb`
Matrículas	http://localhost:8083/h2-console	`jdbc:h2:mem:matriculasdb`

> **Nota:** Usuario: `sa`, Password: vacío

📡 API Endpoints

Todos los endpoints están versionados con `/api/v1/` y disponibles a través del **API Gateway** (puerto 8080).

👨‍🎓 Estudiantes (`/api/v1/students`)

Método	Endpoint	Descripción	Body
`GET`	`/api/v1/students`	Listar todos los estudiantes	-
`GET`	`/api/v1/students/{id}`	Obtener estudiante por ID	-
`POST`	`/api/v1/students`	Crear nuevo estudiante	`StudentCreateReques`
`PATCH`	`/api/v1/students/{id}/status`	Actualizar estado	`StatusUpdateRequest`

Ejemplos:

```
# Crear estudiante
curl -X POST http://localhost:8080/api/v1/students \
```

```
  -H "Content-Type: application/json" \
  -d '{
    "firstName": "Juan",
    "lastName": "Pérez",
    "email": "juan.perez@universidad.edu"
  }'

# Listar todos
curl http://localhost:8080/api/v1/students

# Obtener por ID
curl http://localhost:8080/api/v1/students/1

# Cambiar estado a INACTIVE
curl -X PATCH http://localhost:8080/api/v1/students/1/status \
  -H "Content-Type: application/json" \
  -d '{"status": "INACTIVE"}'
```

Estados de Estudiante: `ACTIVE`, `INACTIVE`, `GRADUATED`, `SUSPENDED`

📚 Cursos (`/api/v1/courses`)

Método	Endpoint	Descripción	Body
`GET`	`/api/v1/courses`	Listar todos los cursos	-
`GET`	`/api/v1/courses/{id}`	Obtener curso por ID	-
`POST`	`/api/v1/courses`	Crear nuevo curso	`CourseCreateReques`
`PATCH`	`/api/v1/courses/{id}/status`	Actualizar estado	`StatusUpdateReques`
`PATCH`	`/api/v1/courses/{id}/capacity`	Actualizar capacidad	`CapacityUpdateRequ`

Ejemplos:

```
# Crear curso
curl -X POST http://localhost:8080/api/v1/courses \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Programación Orientada a Objetos",
    "description": "Fundamentos de POO con Java 21",
    "capacity": 30
  }'

# Listar todos
curl http://localhost:8080/api/v1/courses

# Obtener por ID
```

```
curl http://localhost:8080/api/v1/courses/1

# Cambiar estado
curl -X PATCH http://localhost:8080/api/v1/courses/1/status \
  -H "Content-Type: application/json" \
  -d '{"status": "INACTIVE"}'

# Actualizar capacidad
curl -X PATCH http://localhost:8080/api/v1/courses/1/capacity \
  -H "Content-Type: application/json" \
  -d '{"capacity": 40}'
```

Estados de Curso: `ACTIVE`, `INACTIVE`, `FINISHED`

📝 Matrículas (`/api/v1/enrollments`)

Método	Endpoint	Descripción	Body
`GET`	`/api/v1/enrollments`	Listar todas las matrículas	-
`GET`	`/api/v1/enrollments?studentId={id}`	Filtrar por estudiante	-
`GET`	`/api/v1/enrollments?courseId={id}`	Filtrar por curso	-
`GET`	`/api/v1/enrollments/{id}`	Obtener matrícula por ID	-
`POST`	`/api/v1/enrollments`	Crear nueva matrícula	`EnrollmentCreateRequest`

Ejemplos:

```
# Crear matrícula (requiere estudiante y curso existentes)
curl -X POST http://localhost:8080/api/v1/enrollments \
  -H "Content-Type: application/json" \
  -d '{
    "studentId": 1,
    "courseId": 1
  }'

# Listar todas
curl http://localhost:8080/api/v1/enrollments

# Filtrar por estudiante
curl "http://localhost:8080/api/v1/enrollments?studentId=1"

# Filtrar por curso
curl "http://localhost:8080/api/v1/enrollments?courseId=1"
```

```
# Obtener por ID
curl http://localhost:8080/api/v1/enrollments/1
```

Validaciones al crear matrícula: 1. ✅ Estudiante debe existir y estar `ACTIVE` 2. ✅ Curso debe existir y estar `ACTIVE` 3. ✅ Curso debe tener capacidad disponible 4. ✅ No debe existir matrícula duplicada (mismo estudiante + curso)

🧪 Testing

Resumen de Tests

Módulo	Tests Unitarios	Tests Integración	Total
ms-estudiantes	8	5	13
ms-cursos	5	6	11
ms-matriculas	6	6	12
Total	**19**	**17**	**36**

Ejecutar Tests

```
# Todos los tests
mvn test

# Solo tests de un módulo
mvn test -pl ms-estudiantes

# Tests de integración
mvn verify

# Con reporte de cobertura
mvn test jacoco:report
```

Ver Reportes de Cobertura

Después de ejecutar `mvn test jacoco:report`, los reportes están en:

```
ms-estudiantes/target/site/jacoco/index.html
ms-cursos/target/site/jacoco/index.html
ms-matriculas/target/site/jacoco/index.html
```

Estructura de Tests

```
src/test/java/com/academic/{servicio}/
├── controller/
│   └── {Entity}ControllerTest.java      # Tests unitarios del
controller
```

```
├── service/
│   └── {Entity}ServiceTest.java        # Tests unitarios del service
└── integration/
    └── {Entity}IntegrationTest.java     # Tests de integración E2E
```

💻 Frontend

El proyecto incluye un frontend de ejemplo construido con **React + Vite + Tailwind CSS**.

Iniciar Frontend

```
cd frontend-example
npm install      # Solo la primera vez
npm run dev      # Iniciar servidor de desarrollo
```

Acceder a http://localhost:5173

Características del Frontend

- 📋 **Gestión de Estudiantes**: Crear, listar y cambiar estado
- 📚 **Gestión de Cursos**: Crear y listar cursos
- 📝 **Gestión de Matrículas**: Inscribir estudiantes en cursos
- 🎨 **UI moderna** con Tailwind CSS
- 🔄 **Comunicación con API Gateway** en puerto 8080

Estructura del Frontend

```
frontend-example/
├── src/
│   ├── api/
│   │   └── index.js          # Cliente API (fetch)
│   ├── components/
│   │   ├── Students.jsx      # Componente de estudiantes
│   │   ├── Courses.jsx       # Componente de cursos
│   │   └── Enrollments.jsx   # Componente de matrículas
│   ├── App.jsx               # Componente principal con tabs
│   └── main.jsx              # Entry point
├── package.json
└── vite.config.js
```

🐳 Docker

Archivos de Configuración

```
├── docker-compose.yml          # Orquestación de servicios
├── api-gateway/Dockerfile      # Imagen del Gateway
├── ms-estudiantes/Dockerfile   # Imagen de Estudiantes
├── ms-cursos/Dockerfile        # Imagen de Cursos
├── ms-matriculas/Dockerfile    # Imagen de Matrículas
└── .dockerignore               # Archivos ignorados
```

Comandos Docker

```
# Construir e iniciar todos los servicios
docker-compose up --build

# Iniciar en background
docker-compose up -d

# Ver logs
docker-compose logs -f

# Ver logs de un servicio específico
docker-compose logs -f ms-estudiantes

# Detener servicios
docker-compose down

# Detener y eliminar volúmenes
docker-compose down -v
```

Puertos Expuestos

Servicio	Puerto Host	Puerto Container
api-gateway	8080	8080
ms-estudiantes	8081	8081
ms-cursos	8082	8082
ms-matriculas	8083	8083

📖 Documentación

Documentos del Proyecto

Documento	Descripción	Ubicación
README.md	Este documento	`./README.md`
PLANNING.md	Plan de proyecto y arquitectura	`./PLANNING.md`
DESIGN.md	Diseño técnico detallado	`./docs/DESIGN.md`
SUSTENTACION.md	Guía para sustentación	`./docs/SUSTENTACION.md`

Diagramas UML (PlantUML)

Diagrama	Descripción	Archivo
Arquitectura	Visión general del sistema	`docs/diagrams/architecture.puml`
Clases	Diagrama de clases UML	`docs/diagrams/class-diagram.puml`
Secuencia	Flujo de creación de matrícula	`docs/diagrams/sequence-diagram.puml`

Los diagramas exportados en PNG están en: `docs/diagrams/output/`

Colección Postman

Ubicación: `docs/postman/academic-system.postman_collection.json`

Contiene **12 requests** organizados por servicio:

```
📁 Academic System API
├── 📁 Students (4 requests)
│   ├── Create Student
│   ├── Get All Students
│   ├── Get Student by ID
│   └── Update Student Status
├── 📁 Courses (5 requests)
│   ├── Create Course
│   ├── Get All Courses
│   ├── Get Course by ID
│   ├── Update Course Status
│   └── Update Course Capacity
└── 📁 Enrollments (3 requests)
    ├── Create Enrollment
    ├── Get All Enrollments
    └── Get Enrollment by ID
```

Importar en Postman: 1. Abrir Postman 2. File → Import 3. Seleccionar `docs/postman/academic-system.postman_collection.json`

🔍 Observabilidad

Health Checks

Todos los servicios exponen el endpoint `/actuator/health`:

```
# Verificar salud de todos los servicios
curl -s http://localhost:8080/actuator/health | jq
curl -s http://localhost:8081/actuator/health | jq
curl -s http://localhost:8082/actuator/health | jq
curl -s http://localhost:8083/actuator/health | jq
```

Respuesta esperada:

```
{
  "status": "UP"
}
```

Correlation ID

Todas las peticiones incluyen un header `X-Correlation-Id` para **trazabilidad distribuida**:

1. El cliente envía una petición al Gateway
2. El Gateway genera un `X-Correlation-Id` único (UUID)
3. El ID se propaga a todos los microservicios downstream
4. Todos los logs incluyen el Correlation ID

Logs

Los logs se almacenan en la carpeta `logs/`:

```
# Ver todos los logs en tiempo real
tail -f logs/*.log

# Ver logs de un servicio específico
tail -f logs/ms-estudiantes.log
```

🛠️ Troubleshooting

Puerto ya en uso

```
# Encontrar proceso usando el puerto
lsof -i :8080

# Matar proceso
kill -9 <PID>

# O usar el script de stop
./stop-all.sh
```

Servicio no inicia

```
# Ver logs del servicio
tail -100 logs/ms-{servicio}.log

# Verificar que las dependencias compilaron
mvn clean install -pl common,ms-{servicio}
```

Error de conexión entre servicios

```
# Verificar que todos los servicios están UP
./status.sh

# Verificar conectividad
curl http://localhost:8081/actuator/health
curl http://localhost:8082/actuator/health
curl http://localhost:8083/actuator/health
```

Base de datos H2

```
# Acceder a la consola H2
# URL: http://localhost:808X/h2-console
# JDBC URL: jdbc:h2:mem:{nombredb}
# User: sa
# Password: (vacío)
```

📊 Métricas del Proyecto

Métrica	Valor
Líneas de código	~3,500
Archivos Java	~50
Módulos Maven	5

Métrica	Valor
Tests totales	36
Cobertura estimada	>70%
Endpoints REST	12
Patrones de diseño	3 (Strategy, Template Method, Repository)

🚀 Mejoras Futuras

- [ ] Agregar **Spring Security** con JWT
- [ ] Implementar **Service Discovery** (Eureka)
- [ ] Agregar **Config Server** centralizado
- [ ] Implementar **Circuit Breaker** (Resilience4j)
- [ ] Migrar a **PostgreSQL** para producción
- [ ] Agregar **Swagger/OpenAPI** para documentación automática
- [ ] Implementar **mensajería asíncrona** (RabbitMQ/Kafka)
- [ ] Agregar **métricas** con Micrometer + Prometheus
- [ ] Implementar **caching** con Redis

👨‍💻 Autores

Diego López

Proyecto desarrollado como parte de la **Actividad de Homologación** para el curso de Programación Orientada a Objetos.

📝 Licencia

Este proyecto es de uso académico. Desarrollado para la **Universidad** como demostración de competencias en:

- Programación Orientada a Objetos
- Principios SOLID
- Patrones de Diseño
- Arquitectura de Microservicios
- Java 21 y Spring Boot 3

Hecho con ❤️ usando Java 21 + Spring Boot 3

DL SOFT

🚀 DL Soft

Creamos Experiencias Digitales Inteligentes

Diseñamos y desarrollamos soluciones de software a medida que se adaptan a tus necesidades y ayudan a tu negocio a crecer.

WEBSITE DLSOFT.DEV | WHATSAPP | LINKEDIN | INSTAGRAM | GITHUB

🛠️ Lo Que Construimos

Apps Móviles	Aplicaciones Web	Sistemas Backend	Soluciones IA
Flutter & React Native	React & Next.js	Node.js & Python	Machine Learning

📊 Nuestros Números

Proyectos Entregados 30+ | Clientes Satisfechos 30+ | Años de Experiencia 5+ | Satisfacción 100%

🏭 Industrias que Transformamos

⚖️ Legal & Notarial • 🏥 Salud • 🛡️ Seguridad • 📦 Logística • 💰 Fintech • ⚽ Deportes • 🌍 Viajes • 🏘️ Comunidades • 🌸 Agricultura

---

¿Tienes un proyecto en mente?

📩 CONTÁCTANOS | CONSULTA GRATIS

DL SOFT TECHNOLOGIES S.A.S.

NIT: 902024441-0 • Bucaramanga, Colombia 🇨🇴

📧 admin@dlsoft.dev • 📱 +1 (703) 856-2045

© DL SOFT TECHNOLOGIES S.A.S. Todos los derechos reservados.